

МИНОБРНАУКИ РОССИИ САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ «ЛЭТИ» ИМ. В.И. УЛЬЯНОВА

Кафедра вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6

по дисциплине: “Алгоритмы и структуры данных”

Тема: «Поддержка обработки исключительных ситуаций»

Студенты гр. 4312

Благинин Ф. М.

Вишняков Д. М.

Устинов В. А.

Преподаватель

Аббас Саддам Ахмед

Мохаммед

Санкт-Петербург
2026

Цель работы

Добавить в программу механизм обработки исключений, научиться выявлять и обрабатывать ошибки разных типов, обеспечить устойчивость программы к сбоям

1. Индивидуальное задание

По варианту 36:

Фигура: полукруг

Свойства: поддерживает поворот и отражение;

Позиции на итоговом рисунке: 1, 4, 5 и 12 (Рис. 1.3).

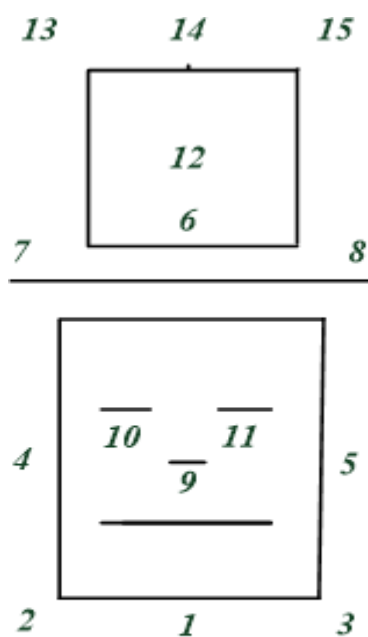


Рис. 1.3. Позиции
для присоединения
дополнительных фигур

2. Добавленные классы исключений

Таблица 1. Классы исключений

Класс	Назначение
ShapeException	Базовый класс для всех исключений фигур, унаследован от <code>std::exception</code>
InvalidParameter	Исключение при некорректных параметрах (отрицательный радиус, нулевая длина и тд)
OutOfScreen	Исключение при выходе фигуры за границы экрана

3. Места размещения throw

В классе `h_circle` переопределены следующие функции-члены базового класса `shape`:

Таблица 2. Переопределенные функции

Место в коде	Условие	Тип исключения
Конструктор <code>h_circle</code>	<code>rd <= 0</code>	<code>InvalidParameter</code>
Конструктор <code>line</code>	<code>L <= 0</code> или точки совпадают	<code>InvalidParameter</code>
Конструктор <code>rectangle</code>	ширина или высота <code><= 0</code>	<code>InvalidParameter</code>
Функция <code>resize()</code>	<code>factor <= 0</code>	<code>InvalidParameter</code>
Функция <code>move()</code>	после перемещения проверка <code>check_bounds()</code>	<code>OutOfScreen</code>
Функции поворота/отражения	после трансформации проверка <code>check_bounds()</code>	<code>OutOfScreen</code>
Функция <code>check_bounds()</code>	любая точка фигуры за границами экрана	<code>OutOfScreen</code>

4. Блоки контроля

Для обработки исключений добавлены две вспомогательные функции:

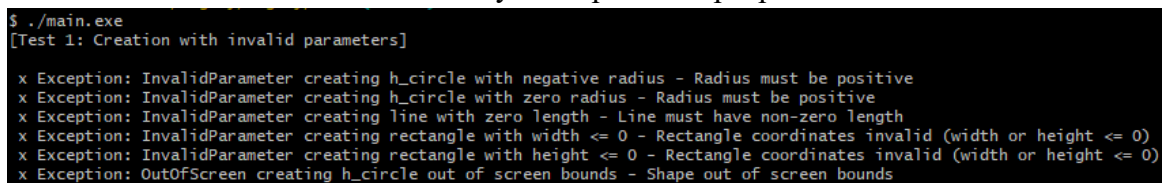
`safe_create` создает объект фигуры в блоке `try/catch`. Если конструктор выбрасывает исключение, фигура не добавляется в список, вместо нее создается `error_marker` в примерном месте предполагаемого расположения.

`safe_execute` выполняет операцию над фигурой (поворот, перемещение и т.д.) в блоке `try/catch`. При возникновении исключения выводится сообщение, а фигура остается в предыдущем состоянии.

5. Примеры работы

5.1 Тест 1. Создание с некорректными параметрами

Результат работы программы:



```
$ ./main.exe
[Test 1: Creation with invalid parameters]
x Exception: InvalidParameter creating h_circle with negative radius - Radius must be positive
x Exception: InvalidParameter creating h_circle with zero radius - Radius must be positive
x Exception: InvalidParameter creating line with zero length - Line must have non-zero length
x Exception: InvalidParameter creating rectangle with width <= 0 - Rectangle coordinates invalid (width or height <= 0)
x Exception: InvalidParameter creating rectangle with height <= 0 - Rectangle coordinates invalid (width or height <= 0)
x Exception: OutOfScreen creating h_circle out of screen bounds - Shape out of screen bounds
```

Рис 5.1

Были попытки создать:

- полукруг с отрицательным радиусом
- полукруг с нулевым радиусом
- линию нулевой длины
- прямоугольник с отрицательной шириной
- полукруг за пределами экрана

Все эти попытки вызвали исключения, которые были перехвачены.

5.2 Тест 2. Нормальное создание фигуры

[Test 2: Normal creation of basic shapes]

```
tbY hat created successfully
tbY brim created successfully
tbY earL created successfully
tbY earR created successfully
tbY tie created successfully
tbY shishak created successfully
tbY face created successfully
```

[First screen]

[illegible]

Puc 5.2

Созданы основные фигуры для сборки: шляпа, поля, два полукруга для ушей, галстук, шишак, лицо. Все созданы успешно.

5.3 Тест 3. Исключения при трансформациях

[illegible]

Рис. 5.3

Все исключения перехвачены, фигуры, вызвавшие ошибки, сохранили свое предыдущее состояние.

5.4 Тест 4. Исключения при сборке

[illegible]

Puc 5.4

Выполнена сборка изображения. При попытке разместить слишком большой полукруг слева от лица возникло исключение OutOfScreen фигура не была размещена. Остальные фигуры встали на свои места.

6. Вывод

В ходе работы в программу добавили обработку исключений. Создали три класса ошибок: `ShapeException` (общий), `InvalidParameter` (для неправильных параметров) и `OutOfScreen` (для выхода за экран).

Исключения генерируются в конструкторах (если радиус отрицательный или линия нулевая) и в функциях `move`, `resize`, `rotate` (если после изменения фигура уходит за экран). Проверку делает функция `check_bounds`.

Обработку сделали в `main` через функции `safe_create` и `safe_execute`. Если ошибка в конструкторе то фигура не создается, вместо нее рисуется крестик. Если ошибка при трансформации то фигура остается как была.

Протестировали: создание с отрицательным радиусом, увеличение в 50 раз, уход за экран. Все ошибки поймались, программа не упала. Картинка собралась, кривые фигуры заменились крестиками.

Программа стала устойчивее, исключения не мешают основной работе. Задача выполнена.

ПРИЛОЖЕНИЕ

А. Файл **shape.h**

```
#include "screen.h"
#include <list>
#include <iostream>
#include <exception>
#include <string>
#include <cmath>
using std::list;

// == Exception Classes ==
class ShapeException : public std::exception {
    std::string msg;
public:
    ShapeException(const char* m) : msg(m) {}
    const char* what() const noexcept override { return msg.c_str(); }
};

class InvalidParameter : public ShapeException {
public:
    InvalidParameter(const char* m) : ShapeException(m) {}
};

class OutOfScreen : public ShapeException {
public:
    OutOfScreen(const char* m) : ShapeException(m) {}
};

//==1. Screen support as a character matrix ==
char screen[YMAX][XMAX];
enum color { black = '*', white = '.' };

void screen_init( )
```

```

{
    for (auto y = 0; y < YMAX; ++y)
        for (auto &x : screen[y]) x = white;
}

void screen_destroy( )
{
    for (auto y = 0; y < YMAX; ++y)
        for (auto &x : screen[y]) x = black;
}

bool on_screen(int a, int b)
{ return 0 <= a && a < XMAX && 0 <= b && b < YMAX; }

void put_point(int a, int b)
{ if (on_screen(a,b)) screen[b][a] = black; }

void put_line(int x0, int y0, int x1, int y1)
{
    int dx = 1;
    int a = x1 - x0; if (a < 0) dx = -1, a = -a;
    int dy = 1;
    int b = y1 - y0; if (b < 0) dy = -1, b = -b;
    int two_a = 2*a;
    int two_b = 2*b;
    int xcrit = -b + two_a;
    int eps = 0;
    for (;;) {
        put_point(x0, y0);
        if (x0 == x1 && y0 == y1) break;
        if (eps <= xcrit) x0 += dx, eps += two_b;
        if (eps >= a || a < b) y0 += dy, eps -= two_a;
    }
}

void screen_clear( ) { screen_init( ); }

void screen_refresh( )
{
    for (int y = YMAX-1; 0 <= y; --y) {
        for (auto x : screen[y])
            std::cout << x;
        std::cout << '\n';
    }
}

```

```

    }
}

```

//== 2. Shape Library ==

```

struct shape {
    static list<shape*> shapes;
    shape() { shapes.push_back(this); }
    virtual point north() const = 0;
    virtual point south() const = 0;
    virtual point east() const = 0;
    virtual point west() const = 0;
    virtual point neast() const = 0;
    virtual point seast() const = 0;
    virtual point nwest() const = 0;
    virtual point swest() const = 0;
    virtual void draw() = 0;
    virtual void move(int, int) = 0;
    virtual void resize(double) = 0;
    virtual ~shape() { shapes.remove(this); }
};

```

```
list<shape*> shape::shapes;
```

```

void shape_refresh()
{
    screen_clear();
    for (auto p : shape::shapes) p->draw();
    screen_refresh();
}

```

```

void check_bounds(const shape& s) {
    point n = s.north();
    point s_ = s.south();
    point e = s.east();
    point w = s.west();

    point ne = s.neast();
    point nw = s.nwest();
    point se = s.seast();
    point sw = s.swest();

    if (w.x < 0 || e.x >= XMAX || s_.y < 0 || n.y >= YMAX ||
        ne.x < 0 || ne.x >= XMAX || ne.y < 0 || ne.y >= YMAX ||

```

```

        nw.x < 0 || nw.x >= XMAX || nw.y < 0 || nw.y >= YMAX ||
        se.x < 0 || se.x >= XMAX || se.y < 0 || se.y >= YMAX ||
        sw.x < 0 || sw.x >= XMAX || sw.y < 0 || sw.y >= YMAX)
        throw OutOfScreen("Shape out of screen bounds");
    }

class error_marker : public shape {
    point center;
public:
    error_marker(point c) : center(c) {
        if (center.x < 2) center.x = 2;
        if (center.x >= XMAX-2) center.x = XMAX-3;
        if (center.y < 2) center.y = 2;
        if (center.y >= YMAX-2) center.y = YMAX-3;
    }
    point north() const override { return point(center.x, center.y+1); }
    point south() const override { return point(center.x, center.y-1); }
    point east() const override { return point(center.x+1, center.y); }
    point west() const override { return point(center.x-1, center.y); }
    point neast() const override { return point(center.x+1, center.y+1); }
    point seast() const override { return point(center.x+1, center.y-1); }
    point nwest() const override { return point(center.x-1, center.y+1); }
    point swest() const override { return point(center.x-1, center.y-1); }
    void draw() override {
        put_line(center.x - 2, center.y, center.x + 2, center.y);
        put_line(center.x, center.y - 2, center.x, center.y + 2);
    }
    void move(int dx, int dy) override { center.x += dx; center.y += dy; }
    void resize(double) override {}
};

class rotatable : virtual public shape {
protected:
    enum class rotated { left, no, right };
    rotated state;
public:
    rotatable(rotated r = rotated::no) : state(r) { }
    void rotate_left() { state = rotated::left; }
    void rotate_right() { state = rotated::right; }
};

class reflectable : virtual public shape {
protected:

```

```

    bool hor, vert;
public:
    reflectable(bool h = false, bool v = false) : hor(h), vert(v) { }
    void flip_horizontally() { hor = !hor; }
    void flip_vertically() { vert = !vert; }
};

class line : public shape {
protected:
    point w, e;
public:
    line(point a, point b) : w(a), e(b) {
        if (a.x == b.x && a.y == b.y)
            throw InvalidParameter("Line must have non-zero length");
        check_bounds(*this);
    }

    line(point a, int L) : w(point(a.x + L - 1, a.y)), e(a) {
        if (L <= 0) throw InvalidParameter("Line length must be positive");
        if (L > XMAX) throw InvalidParameter("Line too long for screen");
        check_bounds(*this);
    }

    point north( ) const { return point((w.x+e.x)/2, e.y<w.y? w.y : e.y); }
    point south( ) const { return point((w.x+e.x)/2, e.y<w.y? e.y : w.y); }
    point east( ) const { return point(e.x<w.x? w.x : e.x, (w.y+e.y)/2); }
    point west( ) const { return point(e.x<w.x? e.x : w.x, (w.y+e.y)/2); }
    point neast( ) const { return point(w.x<e.x? e.x : w.x, e.y<w.y? w.y : e.y); }
    point seast( ) const { return point(w.x<e.x? e.x : w.x, e.y<w.y? e.y : w.y); }
    point nwest( ) const { return point(w.x<e.x? w.x : e.x, e.y<w.y? w.y : e.y); }
    point swest( ) const { return point(w.x<e.x? w.x : e.x, e.y<w.y? e.y : w.y); }

    void move(int a, int b) {
        w.x += a; w.y += b; e.x += a; e.y += b;
        check_bounds(*this);
    }

    void draw( ) { put_line(w, e); }

    void resize(double d) {
        if (d <= 0) throw InvalidParameter("Resize factor must be positive");
        e.x = w.x + static_cast<int>((e.x - w.x) * d);
        e.y = w.y + static_cast<int>((e.y - w.y) * d);

```

```

        check_bounds(*this);
    }
};

class rectangle : public rotatable {
protected:
    point sw, ne;
public:
    rectangle(point a, point b) : sw(a), ne(b) {
        if (a.x >= b.x || a.y >= b.y)
            throw InvalidParameter("Rectangle coordinates invalid (width or height <=
0)");
        check_bounds(*this);
    }

    point north( ) const { return point((sw.x + ne.x) / 2, ne.y); }
    point south( ) const { return point((sw.x + ne.x) / 2, sw.y); }
    point east( ) const { return point(ne.x, (sw.y + ne.y) / 2); }
    point west( ) const { return point(sw.x, (sw.y + ne.y) / 2); }
    point neast( ) const { return ne; }
    point seast( ) const { return point(ne.x, sw.y); }
    point nwest( ) const { return point(sw.x, ne.y); }
    point swest( ) const { return sw; }

    void rotate_right( ) {
        int w = ne.x - sw.x, h = ne.y - sw.y;
        sw.x = ne.x - h * 2;
        ne.y = sw.y + w / 2;
        check_bounds(*this);
    }

    void rotate_left( ) {
        int w = ne.x - sw.x, h = ne.y - sw.y;
        ne.x = sw.x + h * 2;
        ne.y = sw.y + w / 2;
        check_bounds(*this);
    }

    void move(int a, int b) {
        sw.x += a; sw.y += b; ne.x += a; ne.y += b;
        check_bounds(*this);
    }
}

```

```

void resize(double d) {
    if (d <= 0) throw InvalidParameter("Resize factor must be positive");
    ne.x = sw.x + static_cast<int>((ne.x - sw.x) * d);
    ne.y = sw.y + static_cast<int>((ne.y - sw.y) * d);
    check_bounds(*this);
}

void draw( ) {
    put_line(nwest( ), ne); put_line(ne, seast( ));
    put_line(seast( ), sw); put_line(sw, nwest( ));
}
};

```

В. Файл shape.cpp

```

#include <locale.h>
#include <iostream>
#include <cmath>
#include <algorithm>
#include <memory>
#include <vector>
#include <limits>
#include "shape.h"

class h_circle : public rotatable, public reflectable {
public:
    double angle;
    point center;
    int radius;

    h_circle(point a, int rd) : center(a), radius(rd), angle(0) {
        if (rd <= 0) throw InvalidParameter("Radius must be positive");
        check_bounds(*this);
    }

    point north() const override {
        double maxY = -std::numeric_limits<double>::infinity();
        // Check all points on the semicircle arc
        for (double theta = angle; theta <= angle + M_PI + 1e-10; theta += 0.01) {
            double y = center.y + radius * sin(theta);
            if (y > maxY) maxY = y;
        }
    }
};

```

// Also check the center point if it's part of the semicircle


```

    if (sin(angle) >= -1e-10 || sin(angle + M_PI) <= 1e-10) {
        maxY = std::max(maxY, (double)center.y);
    }
    return point(center.x, static_cast<int>(round(maxY)));
}

point south() const override {
    double minY = std::numeric_limits<double>::infinity();
    for (double theta = angle; theta <= angle + M_PI + 1e-10; theta += 0.01) {
        double y = center.y + radius * sin(theta);
        if (y < minY) minY = y;
    }
    if (sin(angle) >= -1e-10 || sin(angle + M_PI) <= 1e-10) {
        minY = std::min(minY, (double)center.y);
    }
    return point(center.x, static_cast<int>(round(minY)));
}

point east() const override {
    double maxX = -std::numeric_limits<double>::infinity();
    for (double theta = angle; theta <= angle + M_PI + 1e-10; theta += 0.01) {
        double x = center.x + radius * cos(theta);
        if (x > maxX) maxX = x;
    }
    if (cos(angle) >= -1e-10 || cos(angle + M_PI) >= -1e-10) {
        maxX = std::max(maxX, (double)center.x + radius);
    }
    if (cos(angle) <= 1e-10 || cos(angle + M_PI) <= 1e-10) {
        maxX = std::max(maxX, (double)center.x - radius);
    }
    return point(static_cast<int>(round(maxX)), center.y);
}

point west() const override {
    double minX = std::numeric_limits<double>::infinity();
    for (double theta = angle; theta <= angle + M_PI + 1e-10; theta += 0.01) {
        double x = center.x + radius * cos(theta);
        if (x < minX) minX = x;
    }
    if (cos(angle) >= -1e-10 || cos(angle + M_PI) >= -1e-10) {
        minX = std::min(minX, (double)center.x + radius);
    }
    if (cos(angle) <= 1e-10 || cos(angle + M_PI) <= 1e-10) {

```

```

        minX = std::min(minX, (double)center.x - radius);
    }
    return point(static_cast<int>(round(minX)), center.y);
}

point neast() const override {
    return point(east().x, north().y);
}

point seast() const override {
    return point(east().x, south().y);
}

point nwest() const override {
    return point(west().x, north().y);
}

point swest() const override {
    return point(west().x, south().y);
}

void draw() override;

void move(int dx, int dy) override {
    center.x += dx;
    center.y += dy;
    check_bounds(*this);
}

void resize(double factor) override {
    if (factor <= 0)
        throw InvalidParameter("Resize factor must be positive");
    radius = static_cast<int>(radius * factor);
    if (radius <= 0)
        throw InvalidParameter("Radius became non-positive after resize");
    check_bounds(*this);
}

void flip_horizontally() {
    if (angle < M_PI) {
        angle = M_PI - angle;
    } else {
        angle = 3*M_PI - angle;
    }
}

```

```

    }
    while (angle < 0) angle += 2*M_PI;
    while (angle >= 2*M_PI) angle -= 2*M_PI;
    check_bounds(*this);
}

void flip_vertically() {
    angle = 2*M_PI - angle;
    while (angle < 0) angle += 2*M_PI;
    while (angle >= 2*M_PI) angle -= 2*M_PI;
    check_bounds(*this);
}

void rotate_right() {
    angle -= M_PI_2;
    while (angle < 0) angle += 2*M_PI;
    check_bounds(*this);
}

void rotate_left() {
    angle += M_PI_2;
    while (angle >= 2*M_PI) angle -= 2*M_PI;
    check_bounds(*this);
}
};

void h_circle::draw() {
    int x = 0;
    int y = radius;
    int delta = 1 - 2 * radius;
    int error;

    while (y >= 0) {
        for (int dx : {x, -x}) {
            for (int dy : {y, -y}) {
                double pointAngle = atan2(dy, dx);
                if (pointAngle < 0) pointAngle += 2 * M_PI;

                double normalizedAngle = pointAngle - angle;
                while (normalizedAngle < 0) normalizedAngle += 2 * M_PI;
                while (normalizedAngle >= 2 * M_PI) normalizedAngle -= 2 * M_PI;

                if (normalizedAngle >= -1e-10 && normalizedAngle <= M_PI + 1e-10) {

```

```

        put_point(center.x + dx, center.y + dy);
    }
}

error = 2 * (delta + y) - 1;
if(delta < 0 && error <= 0) {
    ++x;
    delta += 2 * x + 1;
    continue;
}

error = 2 * (delta - x) - 1;
if(delta > 0 && error > 0) {
    --y;
    delta += 1 - 2 * y;
    continue;
}

++x;
delta += 2 * (x - y);
--y;
}
}

void up(shape& p, const shape& q)
{
    point n = q.north();
    point s = p.south();
    p.move(n.x - s.x, n.y - s.y + 1);
}

void up_center(h_circle& p, const shape& q) {
    point n = q.north();
    point p_center = p.center;
    p.move(n.x - p_center.x, n.y - p_center.y + 1);
}

void down_center(h_circle& p, const shape& q) {
    point s = q.south();
    point p_center = p.center;
    p.move(s.x - p_center.x, s.y - p_center.y - 1);
}

```

```

void right_center(h_circle& p, const shape& q) {
    point e = q.east();
    point p_center = p.center;
    p.move(e.x - p_center.x + 1, e.y - p_center.y);
}

void left_center(h_circle& p, const shape& q) {
    point w = q.west();
    point p_center = p.center;
    p.move(w.x - p_center.x - 1, w.y - p_center.y);
}

class myshape : public rectangle {
    int w, h;
    line l_eye;
    line r_eye;
    line mouth;
public:
    myshape(point, point);
    void draw( );
    void move(int, int);
    void resize(double r) {
        if (r <= 0) throw InvalidParameter("Resize factor must be positive");
        rectangle::resize(r);
        rectangle::move(w*(1-r)*0.5, h*(1-r)*0.5);
    }
    void rotate_left() {}
    void rotate_right() {}
};

myshape :: myshape(point a, point b)
    : rectangle(a, b),
    w(neast( ).x - swest( ).x + 1),
    h(neast( ).y - swest( ).y + 1),
    l_eye(point(swest( ).x + 2, swest( ).y + h * 3 / 4), 2),
    r_eye(point(swest( ).x + w - 4, swest( ).y + h * 3 / 4), 2),
    mouth(point(swest( ).x + 2, swest( ).y + h / 4), w - 4)
{}

void myshape :: draw( )
{
    rectangle :: draw( );
}

```

```

    int a = (swest( ).x + neast( ).x) / 2;
    int b = (swest( ).y + neast( ).y) / 2;
    put_point(point(a, b));
}

void myshape :: move(int a, int b)
{
    rectangle :: move(a, b);
    l_eye.move(a, b); r_eye.move(a, b);
    mouth.move(a, b);
}

template<typename Func, typename... Args>
bool safe_execute(const char* operation_name, Func&& func, Args&&... args) {
    try {
        func(std::forward<Args>(args)...);
        std::cout << " ✓ Success\n";
        return true;
    } catch (const InvalidParameter& e) {
        std::cout << " x Exception: InvalidParameter in " << operation_name
            << " - " << e.what() << std::endl;
    } catch (const OutOfScreen& e) {
        std::cout << " x Exception: OutOfScreen in " << operation_name
            << " - " << e.what() << std::endl;
    } catch (const ShapeException& e) {
        std::cout << " x Exception: ShapeException in " << operation_name
            << " - " << e.what() << std::endl;
    } catch (const std::exception& e) {
        std::cout << " x Exception: std::exception in " << operation_name
            << " - " << e.what() << std::endl;
    }
    return false;
}

template<typename T, typename... Args>
std::unique_ptr<T> safe_create(const char* shape_name, Args&&... args) {
    try {
        auto ptr = std::make_unique<T>(std::forward<Args>(args)...);
        std::cout << " ✓ " << shape_name << " created successfully\n";
        return ptr;
    } catch (const InvalidParameter& e) {
        std::cout << " x Exception: InvalidParameter creating " << shape_name

```

```
<< " - " << e.what() << std::endl;
```

```

    } catch (const OutOfScreen& e) {
        std::cout << " x Exception: OutOfScreen creating " << shape_name
            << " - " << e.what() << std::endl;
    } catch (const ShapeException& e) {
        std::cout << " x Exception: ShapeException creating " << shape_name
            << " - " << e.what() << std::endl;
    } catch (const std::exception& e) {
        std::cout << " x Exception: std::exception creating " << shape_name
            << " - " << e.what() << std::endl;
    }
    return nullptr;
}

int main( )
{
    screen_init( );

    std::cout << "[Test 1: Creation with invalid parameters]\n" << std::endl;

    safe_create<h_circle>("h_circle with negative radius", point(30, 5), -2);
    safe_create<h_circle>("h_circle with zero radius", point(30, 5), 0);
    safe_create<line>("line with zero length", point(20, 9), point(20, 9));
    safe_create<rectangle>("rectangle with width <= 0", point(10, 10), point(5, 15));
    safe_create<rectangle>("rectangle with height <= 0", point(10, 10), point(15, 5));
    safe_create<h_circle>("h_circle out of screen bounds", point(200, 200), 10);

    std::cout << "[Test 2: Normal creation of basic shapes]\n" << std::endl;

    auto hat = safe_create<rectangle>("hat", point(0, 0), point(14, 5));
    auto brim = safe_create<line>("brim", point(20, 9), 17);
    auto earL = safe_create<h_circle>("earL", point(30, 5), 2);
    auto earR = safe_create<h_circle>("earR", point(40, 5), 2);
    auto tie = safe_create<h_circle>("tie", point(50, 5), 2);
    auto shishak = safe_create<h_circle>("shishak", point(60, 5), 2);
    auto face = safe_create<myshape>("face", point(15, 10), point(27, 18));

    std::cout << "[First screen]\n";
    shape_refresh();
    std::cout << "\nPress Enter to continue...\n";
    std::cin.get();

    std::cout << "[Test 3: Exceptions during transformations]\n" << std::endl;

```



```

if (earL) {
    std::cout << "Rotating earL left: ";
    safe_execute("rotate_left earL", [&]() { earL->rotate_left(); });
    std::cout << "Rotating earL left again: ";
    safe_execute("rotate_left earL", [&]() { earL->rotate_left(); });
}

if (tie) {
    std::cout << "Flipping tie horizontally: ";
    safe_execute("flip_horisontally tie", [&]() { tie->flip_horisontally(); });
    std::cout << "Flipping tie horizontally again: ";
    safe_execute("flip_horisontally tie", [&]() { tie->flip_horisontally(); });
}

if (earR) {
    std::cout << "Attempting to resize earR with factor 0: ";
    safe_execute("resize earR factor 0", [&]() { earR->resize(0); });
}

if (earR) {
    std::cout << "Attempting to enlarge earR 50x: ";
    safe_execute("resize earR factor 50", [&]() { earR->resize(50); });
}

if (shishak) {
    std::cout << "Attempting to move shishak out of screen bounds: ";
    safe_execute("move shishak (x+100)", [&]() { shishak->move(100, 0); });
}

if (hat) {
    std::cout << "Attempting to rotate hat right: ";
    safe_execute("rotate_right hat", [&]() { hat->rotate_right(); });
}

std::cout << "[Second screen]\n";
std::cout << "Shapes that caused exceptions preserved their previous state\n";
shape_refresh();
std::cout << "\nPress Enter to continue...\n";
std::cin.get();

std::cout << "[Test 4: Exceptions during shape assembly]\n" << std::endl;

if (brim && face) {

```

```

    std::cout << "Placing brim above face: ";
    safe_execute("up(brim, face)", [&]() { up(*brim, *face); });
}

if (hat && brim) {
    std::cout << "Placing hat above brim: ";
    safe_execute("up(hat, brim)", [&]() { up(*hat, *brim); });
}

if (shishak && hat) {
    std::cout << "Placing shishak above hat: ";
    safe_execute("up_center(shishak, hat)", [&]() { up_center(*shishak, *hat); });
}

std::cout << "Attempting to place large circle left of face: ";
auto test_circle = safe_create<h_circle>("test_circle for left_center", point(10, 10),
30);
if (test_circle && face) {
    safe_execute("left_center(test_circle, face)", [&]() { left_center(*test_circle,
*face); });
}

std::cout << "[Final screen]\n";
shape_refresh();
std::cout << "\nPress Enter to exit...\n";
std::cin.get();

screen_destroy();
}

```